

[DRAFT] Crucible Function Generator Supplemental Information

Daniel M. Kennedy^{*1} and Ghaith Rabadi^{†1}

¹School of Modeling, Simulation and Training, University of Central Florida, Orlando, Florida, USA

August 9, 2026

1 Introduction

This document provides supplemental details for the simulation contained within this repository [1] and detailed in the parent paper: "Crucible: An Adversarial, Semi-Cooperative, Scalable Function Generator for Testing Meta- and Hyper-Heuristic Optimization Algorithms". Below you will find details for the simulation requirements derivation, software architecture, software features, and simulation default configuration.

2 Crucible Simulation Requirements

Table 1 lists the derived requirements for the Crucible simulation to include abbreviated justification for each. The table is split into major conceptual sections that include overall architecture, composition functions, network pathfinding, iso-optimal surfaces, stochasticity, dynamic behavior, adversarial behavior, and semi-cooperative behavior. Most requirements reproduce or extend qualities from the CEC challenges and other optimization testing references described in the parent paper.

Table 1: Simulation requirements and justification.

Req No.	Requirements	Justification
Architecture Requirements		
1	Written in a compiled language	Based on [2]
2	Problem scenarios are random	Based on [3]
3	Results are repeatable	Required for verification and reproducibility
4	Black-box responses	Based on [4] [5] [6]
5	Reports single target network fitness score as a function of inputs	Based on [7]
6	Reports if inputs result in either a feasible or infeasible solution	Support constrained optimization
7	Report the cumulative number of function evaluations in the output	Based on [5] [6]
8	Support for a report log of the simulation scenario definition	Support experiment record keeping
9	Provide the capability to enable and disable features	Support experimentation with specific features

^{*}Corresponding Author: da711491@ucf.edu

[†]Ghaith.Rabadi@ucf.edu

Req No.	Requirement	Justification
10	Provide the capability to set feature parameters	Support experimentation with specific parameters
11	Support for continuous and mixed-integer problems	Based on [8]
12	Support for separable and non-separable inputs	Based on [9] [10] [11] [12] [13] [14] [6]
13	Multi-paradigm problem combining pathfinding and single objective maximization	Based on [7]
14	Support for scalable problem complexity	Based on [9] [10] [11] [13] [5] [6]
15	Ensure that simulation response truth is between zero and one	Provide known solutions based on [15]
16	Ensure that the optimal response is a maximization with value at one	Simplifies scale factor implementations
17	Support for variable model input and response fidelity	Based on [16]
18	Support for niching optimization by providing many optimal solutions	Based on [11] [7] [17]
19	Support for dynamic problems	Based on [5] [17]
20	Support for adversarial behavior	Introduce adversarial surprise in decision making [18] and include unanticipated consequence to decisions [19]
21	Support for semi-cooperative behavior	Exaggeration of stochastic optimization difficulty [20] and inclusion of failure modes [18]
Composition Function Requirements		
22	Create governing functions using basic functions	Based on [9] [8]
23	Create model functions using basic functions	Based on [14] [6] [8]
24	Create model responses by mapping inputs to model functions through governing functions	Based on [9] [8]
25	Create composition functions using a weighted sum of model responses	Based on [9] [11] [4] [14] [6] [17]
26	Support for non-axis aligned composition functions	Based on [21] [13] [14] [5] [6] [17]
27	Support for scalable number of composition functions	Based on [9] [6] [17]
28	Support composition equation landscapes that can be smooth, discontinuous, with narrow or wide basins, shallow or deep basins, multi-modal, flattened, and non-linear	Based on [22] [23] [9] [10] [11] [4] [14] [5] [6] [17]
29	Support for deceptive basins	Based on [24] [5]
30	Support for composition function scaling using basic equations	Enhance Req. 28 and 31
31	Support for variable conditioning between inputs and responses	Based on [13] [5] [8]
32	Create constraint functions using basic equations	Based on [22] [12]
33	Map inputs to constraint functions through governing equations	Based on [22] [8]
34	Support feasible and infeasible input regions	Based on [22] [21] [12] [7] [8]
35	Support for separable and non-separable constraints	Based on [22] [21] [12] [8]
Network Pathfinding Requirements		
36	Support pathfinding through networked nodes	Based on [7]
37	Generate composition functions at each node	Derived from Req. 13
38	Calculate a node single target fitness score from a weighted sum of composition functions	Based on [14] [6]
39	Calculate a network single target fitness score from a weighted sum of node fitness scores in the pathfinding path	Based on [7]

Req No.	Requirement	Justification
40	Support for network path feasibility based on network trajectory	Based on [7]
41	Support for interactions between node importance and network path trajectory	Based on [7]
42	Support network node visitation requirements	Based on [7]
Node Iso-Optimal Surface Requirements		
43	Support for a global iso-optimal surface	Based on [10] [11] [25] [7] [8]
44	Support for disconnected iso-optimal regions	Based on [10] [11] [25] [7] [8]
45	Support for scalable number of local iso-optimal surfaces	Based on [23] [11] [13] [25] [7]
Stochastic Requirements		
46	Support for stochastic inputs	Based on [26] [8]
47	Support for stochastic responses from deterministic inputs	Based on [2] [23] [27] [8]
48	Randomness is seeded	Required for repeatable results
49	Support for stochastic network effects	Based on GPS discussion in [5]
50	Support for stochastic constraints	Based on [8]
51	Support for randomized input mapping to composition functions	Based on [6]
52	Support for stochastic dynamic discrete event timing	Based on [23] [26]
53	Support for stationary and non-stationary distributions	Based on [2] [23] [26]
Dynamic Behavior Requirements		
54	Support for discrete events based on number of simulation iterations	Based on [23] [17]
55	Support for discrete network path configuration change events	Based on GPS discussion in [5]
56	Support for continuously changing basic function landscapes based on number of simulation iterations	Based on [23] [5] [17]
57	Support for continuously changing constraints based on number of simulation iterations	Enhance Req. 56
58	Support discrete objective and constraint landscape change events	Based on [23] [17]
Adversarial Behavior Requirements		
59	Support for moving optimal location as a function of previous fitness score	Introduce adversarial surprise in decision making [18] and include unanticipated consequence to decisions [19]
60	Support for variable frequency of semi-cooperative behavior as a function of previous fitness score	Adversarial enhancement of semi-cooperative effects
Semi-Cooperative Behavior Requirements		
61	Support for partial degradation and complete failure results	Inclusion of failure modes [18] and optimization strategies in the presence of failure [28]
62	Support for deterministic failures	Based on [28]
63	Support for separable and non-separable failure regions	Extension of Req. 12
64	Support for pathfinding based failures	Based on [28]
65	Support for random incorrect fitness result	Exaggeration of stochastic optimization difficulty [20]
66	Support for random incorrect feasibility result	Exaggeration of stochastic optimization difficulty [20]

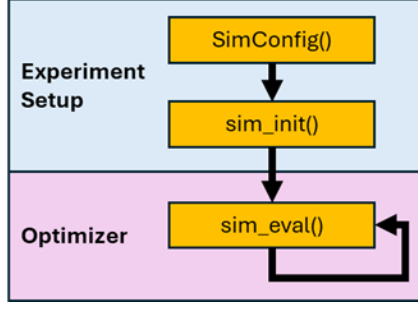


Figure 1: High level relationship of simulation public interfaces for setting up an experiment and evaluating fitness using an optimizer.

3 Simulation Architecture

The Crucible simulation is written in the Julia programming language [29] to provide compiled performance with repeatable environments (Req. 1, 3). Figure 1 provides a high level relationship of the simulation public interfaces when used in an experiment. Simulation configuration is performed by creating a configuration object using the `SimConfig` constructor function. This object is used to enable simulation features and set feature parameters (Req. 9, 10). A comprehensive list of configuration parameters with setpoints and justification is provided in Section 5. Once the configuration object has been constructed the user can override settings as desired. Table 2 provides a description of important simulation structures and public interface functions. The hierarchical relationship between simulation structures is illustrated in Figure 2. The only simulation structure that the user interacts with is the `SimConfig` structure.

Table 2: Crucible Simulation Interface and Structure Descriptions

Name	File	Description
Public Interface		
<code>SimConfig()</code>	<code>config.jl</code>	Returns a default initialized simulation configuration object
<code>sim_init()</code>	<code>initialize.jl</code>	Used to construct the simulation given a configuration object
<code>sim_eval()</code>	<code>evaluate.jl</code>	Used to evaluate fitness for a given input array, network path, and fidelity
Crucible Sim Structures		
<code>SimStruct</code>	<code>sim_obj.jl</code>	Container for scenario, configuration, and adversarial state
<code>SimConfig</code>	<code>config.jl</code>	Contains settings for all configuration parameters
<code>SimCompositionFunction</code>	<code>composition.jl</code>	Composition model mapping, scale spline, and dimension coupling pairs
<code>SimCouplingRegion</code>	<code>coupling.jl</code>	Pair mapping, constraint spline, and skew magnitude
<code>SimFault</code>	<code>fault.jl</code>	Fault threshold and degradation factor
<code>SimFlatten</code>	<code>fault.jl</code>	Location for model response flattening regions
<code>SimInvert</code>	<code>fault.jl</code>	Location for model response inversion regions
<code>SimModel</code>	<code>model.jl</code>	Contains governing and model function splines, dynamic velocity magnitudes, and flattening and inversion data
<code>SimNode</code>	<code>node.jl</code>	Holds node models, composition functions, fault, and scaling spline
<code>SimSpline</code>	<code>spline.jl</code>	Constructor and evaluator for spline basic functions

The `sim_init` function constructs the simulation using the provided configuration object. During construction a log of the scenario features and configuration parameters is written to the filename provided in the log file configuration parameter or the file “logfile.txt” by default (Req. 8). Additional debug figures

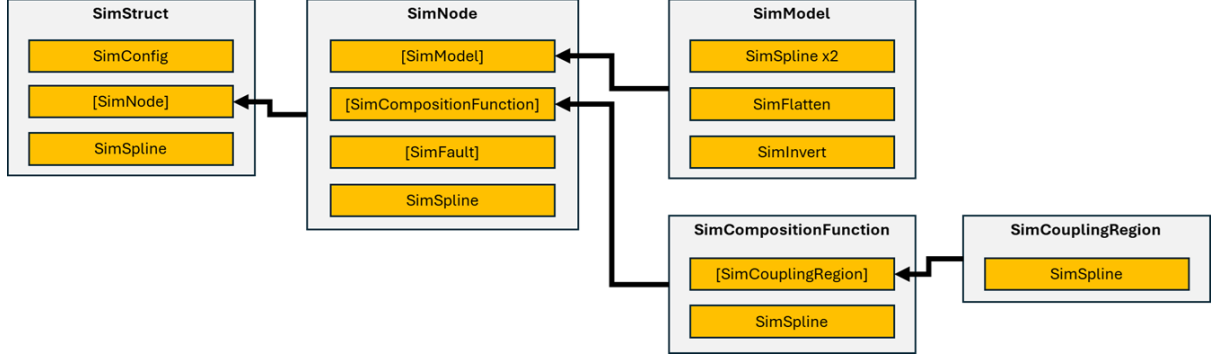


Figure 2: Hierarchical relationship diagram of important simulation structures. Names in brackets indicate arrays.

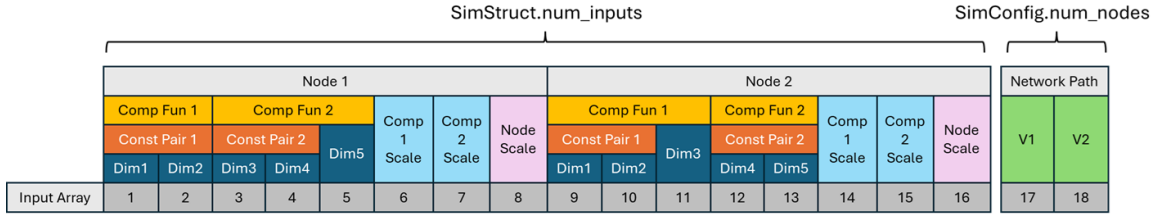


Figure 3: Example input array with sixteen design inputs and two network path inputs. All values are continuous in the range $[0, 1)$. Descriptions of the features in each column are provided in Section 4.

can be created by enabling the `save_debug_figs` option. After the simulation is constructed, the number of design inputs required for evaluating the simulation scenario is stored in the `num_inputs` simulation object parameter.

Scenario level randomization is controlled by seeding an Xoshiro random number stream from the Random package using the scenario seed configuration parameter (Req. 3, 48). This stream is used for constructing the simulation prior to execution (Req. 2). Run level randomization is controlled by seeding another Xoshiro random number stream using the noise seed configuration parameter plus the run iteration (Req. 3, 48).

A simulation run is performed by calling the `sim_eval` function with the simulation object, the design parameters, the desired network path (Req. 13), and a fidelity level (Req. 17). This function reports a single value for the fitness score (Req. 5, 15) in the range $[0, 1)$, with $1.0 - \epsilon$ being optimal, where ϵ is the minimum 64 bit floating point value (Req. 16). In the event of an infeasible solution, the fitness score is reported as NaN (Req. 6). An example simulation input array is provided in Figure 3. Each input value is continuous in the range $[0, 1)$. This example shows two nodes, each with five dimensions. Each node contains two composition functions containing constraint pairs. Each composition function and each node have an additional scaling function input value. The first block of inputs are design values, and the last two inputs are used to determine the attempted network path. Details on all of these features are provided in the sections that follow.

After each simulation run, the simulation object parameter `num_iterations` contains the current run iteration and `num_function_evals` contains an updated cumulative number of spline function evaluations since the object was created (Req. 7). Due to dynamic and adversarial simulation features, each simulation run is coupled to the previous run and cannot be parallelized. The previous run score and number of iterations is stored as simulation state between runs. However, simulations with different starting noise seeds or starting input values can be parallelized to support niching optimization methods and validation of results.

The foundational element for creating landscapes in the simulation are spline basic functions. Spline basic functions are randomly generated splines using the scenario seed (Req. 2). The number of knots in the spline are randomly chosen using the min and max knots input parameters. Knots are added in series using the equation: $y[1] = \text{rand}() - \text{rand}()$; $y[i] = y[i - 1] * \text{knot_feedback} + \text{rand}() - \text{rand}()$. The `knot_feedback` parameter is a value between 0 and 1 that determines how auto-correlated each knot is to the previous knot. When querying a spline function, the input and output ranges are clamped in the

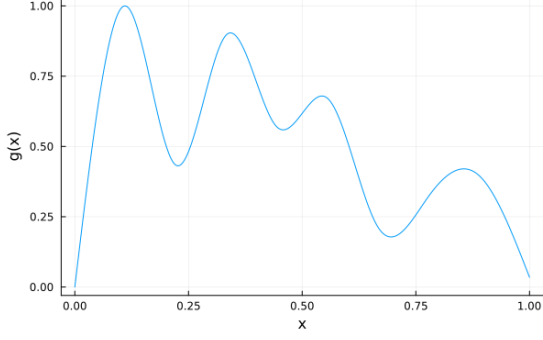


Figure 4: Governing function created using a spline basic function.

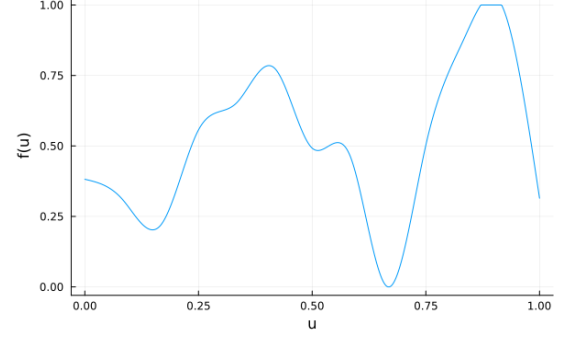


Figure 5: Model function created using a spline basic function.

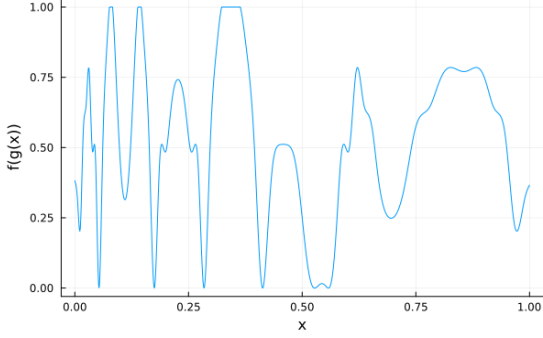


Figure 6: Model response landscape created by mapping design variable input to model function input through a governing function.

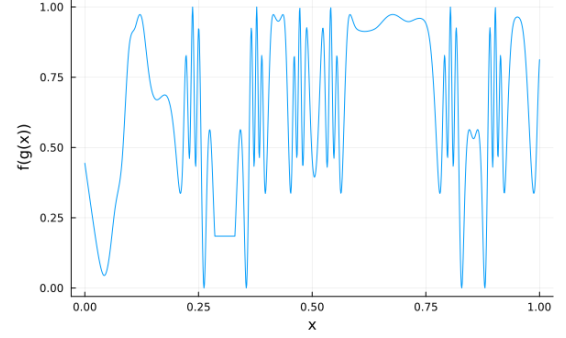


Figure 7: Example model response with many combined features: smooth, discontinuous, narrow and wide basins, shallow and deep basins, multi-modal, flattened, non-linear, and deceptive basins.

range $[0.0, 1.0]$. Spline basic functions are used throughout the simulation for creating governing, model, scaling, constraint, and adversarial functions as described in the following sections.

4 Simulation Features

4.1 Governing Functions and Model Functions

The governing functions and model functions are created using the spline basic function generator (Req. 22, 23). The model response landscape is created by mapping design variable inputs to model function outputs through a governing function (Req. 24). For example, a governing function $g(x)$ (Fig. 4) is used to map input value x to model function $f(u)$ (Fig. 5) resulting in a multi-featured model response landscape $f(g(x))$ (Fig. 6). Figure 7 illustrates an example model response with many combined features: smooth, discontinuous, narrow and wide basins, shallow and deep basins, multi-modal, flattened, and non-linear (Req. 28). This example shows variable conditioning between the input and response (Req. 31), many potential optimal locations with values at 1.0 to support niching optimization methods (Req. 18), as well as multiple deceptive basins near 1.0 (Req. 29). Flattening may occur when a governing or model function is clamped in the range $[0, 1]$, however additional random flattening can be injected using the `enable_flattening` configuration parameter (Fig. 8). Moreover, additional random discontinuity can be injected by randomly inverting regions using the `enable_inversion` configuration parameter (Fig. 9).

4.2 Composition Functions

Composition functions are created by taking the weighted sum of model responses (Req. 25). The number of composition functions and the assignment of model functions to composition functions is

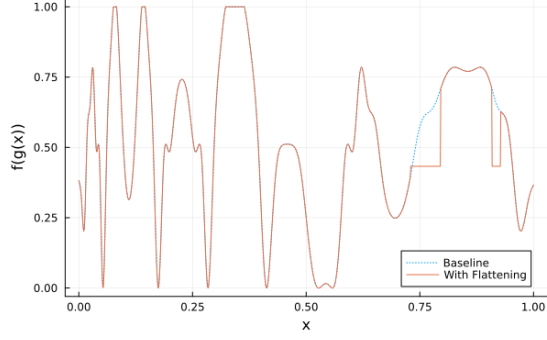


Figure 8: Example random flattening injected using the `enable_flattening` configuration parameter.

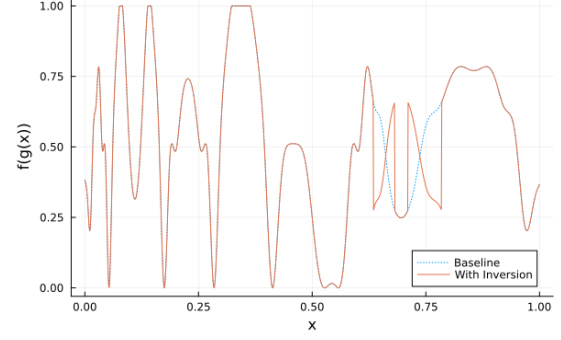


Figure 9: Example random discontinuity injected using the `enable_inversion` configuration parameter.

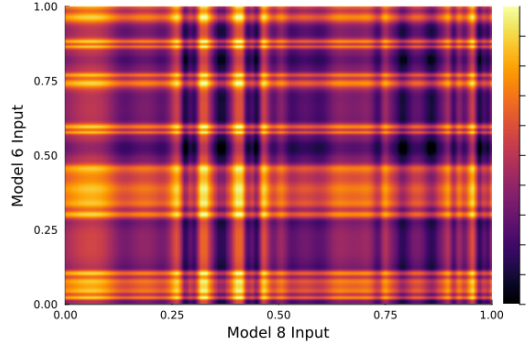


Figure 10: Example composition function response of two models.

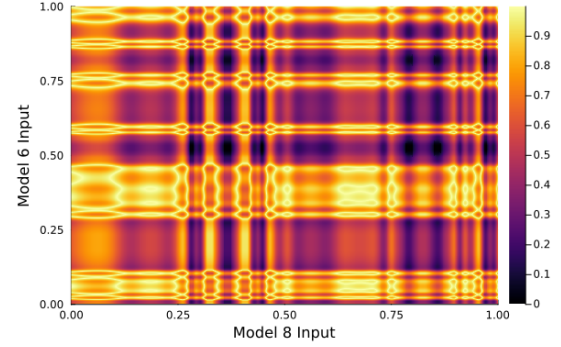


Figure 11: Composition function response with added iso-optimal surface feature.

random based on scenario seed and scaled by number of dimensions (Req. 14, 27). Model function weights are randomly chosen during scenario creation. Randomly assigning model functions to different composition functions creates a mixture of separable and non-separable inputs (Req. 12).

For each composition function, an additional input dimension is created for a scaling effect on the composition function response. This scaling function uses the spline basic function generator. When the simulation is evaluated, this scaling multiplier is applied to the composition function (Req. 30). For each node, an additional input dimension is created using the same process and applied as a scaling factor to the combination of composition functions. This results in $D + C + 1$ inputs per node, where D is the number of dimensions, C is the number of scale factors for composition functions, plus one for the final combined total scaling factor function. The total fitness for each node is the weighted sum of each scaled composition function at that node, scaled by the total scaling factor function (Req. 38).

Figure 10 illustrates an example composition landscape of the sum of two models. Many optimal locations at 1.0 are present to support niching optimization methods (Req. 18). An iso-optimal surface is created by scaling the composition response and inverting any portion that exceeds a value of 1.0. Figure 11 illustrates an iso-optimal surface that provides many disconnected iso-optimal regions (Req. 44) further facilitating niching optimization methods (Req. 18). Iso-optimal surfaces are scaled with the number of composition functions (Req. 45) resulting in a global iso-optimal surface for the combination of composition functions (Req. 43).

4.3 Skewing and Constraints

During the creation of composition functions, pairs of dimensions are randomly coupled to create non-separability and non-axis alignment effects (Req. 12, 26). A skewing effect is created by randomly picking a skewing factor for each input. The first input is skewed as a function of the second input, then the second input is skewed as a function of the skewed first input. Skewing may be positive or negative and is controlled using the `enable_skewing` and `max_skew_magnitude` configuration parameters. Figure 12 illustrates skewing applied to the previous example. Skewed model inputs are wrapped in the range

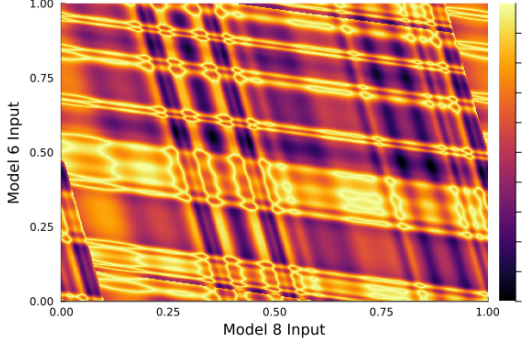


Figure 12: Composition function response with added wrapped skewing feature to create non-separability and non-axis alignment.

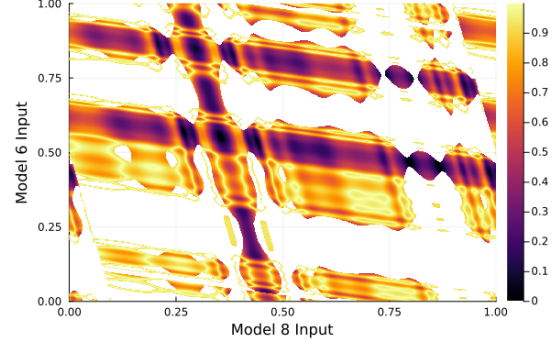


Figure 13: Composition function response with added feasibility constraint feature. White areas correspond to infeasible results.

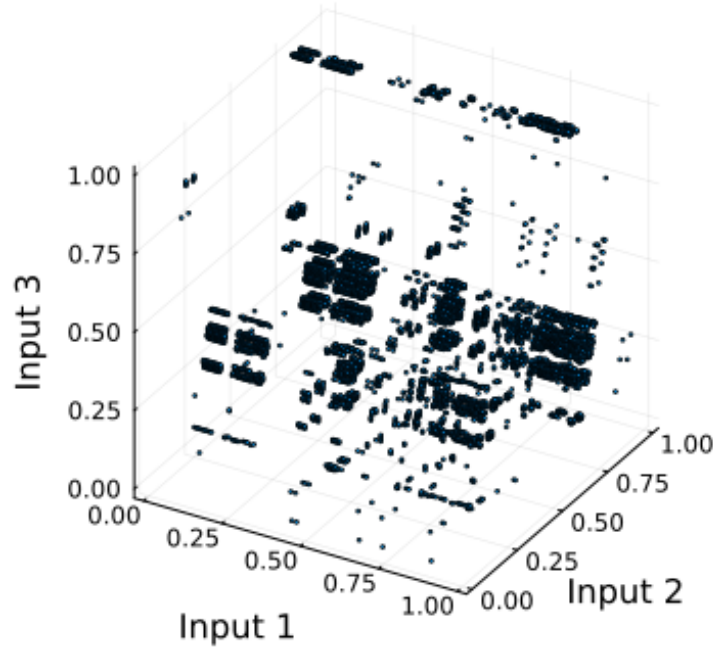


Figure 14: An example iso-optimal surface for a 3-Dimensional composition function with skewing and constraint features. All points displayed have a fitness of approximately 1.0.

$[0, 1)$ to remain valid and prevent regions of the solution landscape from becoming unreachable.

Feasibility constraints are created on coupled composition pairs using a constraint function. The constraint function is created using the spline basic function generator (Req. 32). One of the pair input values is selected as the constraint function input. Feasibility is determined by checking if the other pair input value is above or below (chosen at random for the constraint) of the spline response. The coupled pair input values are transformed using their respective governing functions prior to evaluating the constraint function creating a varied feasibility landscape (Req. 33). Figure 13 builds on the previous skewing example with additional infeasible regions shown in white (Req. 34). Constraints are applied to pairs of inputs resulting in separable and non-separable constraints when more than one constraint is present (Req. 35).

Two margins are included to prevent a scenario in which there are no feasible optimal solutions. The first is an input margin controlled via the `constraint_input_margin` configuration parameter that requires the input variable enter the infeasible region by at least that amount before infeasibility is declared. The second margin is a small margin around optimal locations controlled via the `constraint_optimal_margin` configuration parameter. This margin results in small islands of optimal feasible solutions in Figure 13. A three-dimensional example is provided in Figure 14.

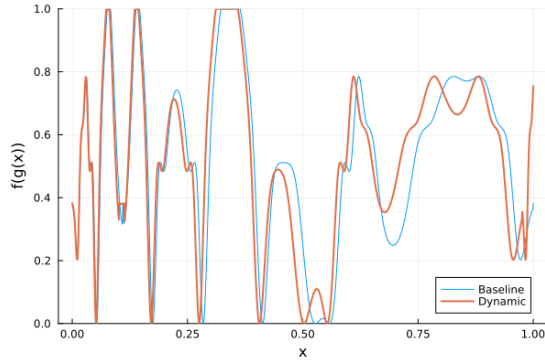


Figure 15: Example model response showing continuous dynamic spline landscape change as a function of simulation iterations.

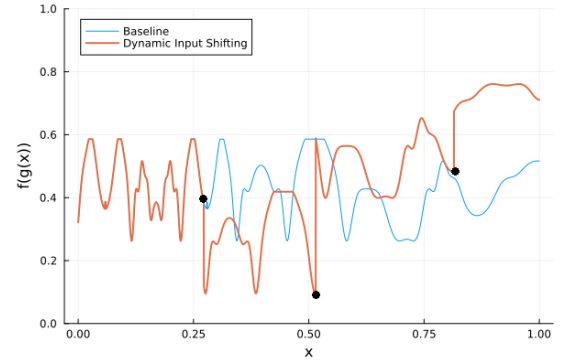


Figure 16: Example model response with discrete dynamic input shift biasing events as a function of simulation iterations.

4.4 Dynamic Features

The simulation provides features for dynamically changing the scenario structure as a function of optimization iterations with the `enable_dynamic` configuration option. During each call to `sim_eval()`, the `num_iterations` counter increments which affects continuous changes and discrete events. Continuous changes are created through the use of a velocity parameter for each input dimension and governing function response (Fig. 15). For a constant input value, each simulation iteration will cause the composition function and constraint landscapes to slowly change (Req. 56, 57). This feature is controlled using the `enable_dynamic_spline`, `dyn_model_inp_vel_mag`, and `dyn_model_gov_vel_mag` configuration parameters.

Multiple dynamic discrete events features are available (Req. 54). Each design input is assigned a random offset during scenario creation using the `enable_input_shift` option. If dynamic features are enabled, the `enable_dynamic_input_shift` option will result in periodic jump shifts for each input offset. The magnitude and frequency of the shifts are controlled via the `dyn_input_shift_mag`, `input_shift_freq_min`, and `dyn_input_shift_freq_max` feature respectively. When the number of iterations reaches a randomly selected threshold between the min and max frequency (Req. 52) all inputs will be offset simultaneously resulting in a new bias in the design inputs. This will result in dynamic jumps in the composition function and constraint landscapes (Req. 58). Figure 16 shows three input shifting events (indicated by black circles) across four-thousand simulation evaluations sweeping across a design input from zero to one.

Each design input is mapped to simulation models using a mapping vector, enabled using the `enable_input_map` option (Req. 51). If dynamic features are enabled as well as the `enable_dynamic_input_map` option, then the mapping of design variables to models will randomly mutate at discrete events (Req. 58). Mutations occur between random pairs of inputs at a random simulation iteration chosen between the frequency ranges defined in the `dyn_input_map_freq_min` and `dyn_input_map_freq_max` configuration parameters. Figure 17 illustrates a dynamic input mapping change event at the 1000th simulation iteration while sweeping the input dimension from zero to one over two-thousand evaluations. Figure 18 shows an example response of the combination of the dynamic continuous input velocity feature and the dynamic discrete input mapping change event.

Lastly, the simulation supports a dynamic discrete event feature to periodically mutate the simulation network when using more than one node (Req. 55). This feature is enabled using the `enable_dynamic_network` option. The frequency of the event is controlled using the `dyn_network_map_freq_min` and `dyn_network_map_freq_max` configuration parameters. The network mutation event swaps two nodes along the nominal path so that the majority of the path remains valid. However, random links between paths are regenerated from scratch at each event (Req. 49).

4.5 Stochastic Features

The simulation provides a variety of stochastic features to challenge optimizers. Noise can be added to the design input values to create uncertainty into the governing functions (Req. 26). This is controlled via the `enable_input_noise` and `input_noise_sigma` configuration parameters. Because the uncertainty is created at the governing function inputs this results in uncertainty at constraint boundaries (Req.

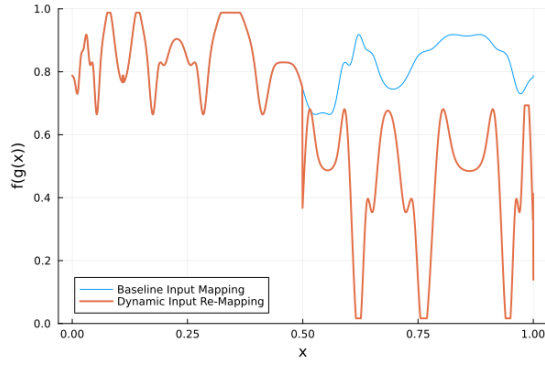


Figure 17: Example model response with discrete design input re-mapping event as a function of simulation iterations.

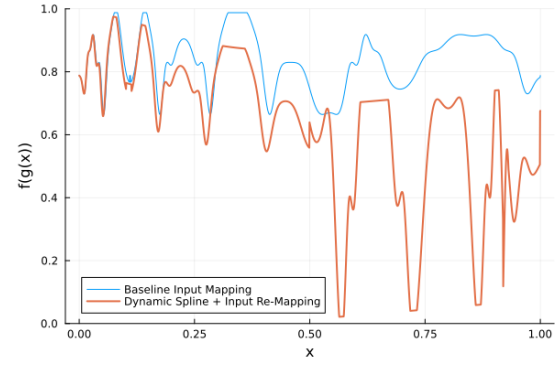


Figure 18: Example model response that combines a discrete input re-mapping event with continuous dynamic spline landscape changes as a function of simulation iterations.

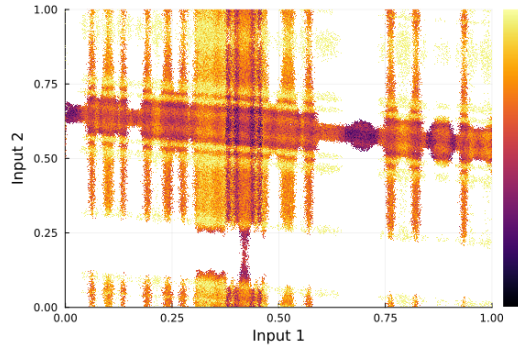


Figure 19: Stochastic composition function response created by noise added to input dimensions.

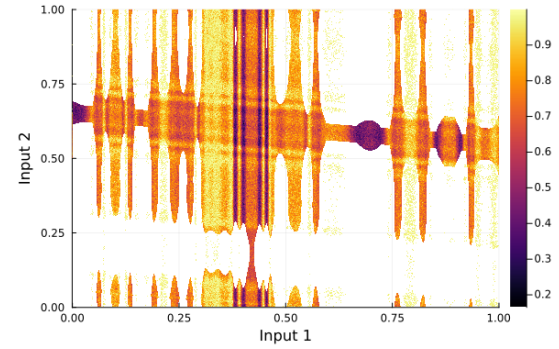


Figure 20: Stochastic composition function response created by noise added to model responses.

50). Additionally, noise can be added to the model response outputs using the `enable_model_noise` and `model_noise_sigma` configuration parameters (Req. 47). Figures 19 and 20 illustrate these respective features.

4.6 Variable Model Fidelity

When evaluating simulation fitness, the optimizer must include an integer parameter for model fidelity (Req. 17). This allows replicates to be performed when determining input and response uncertainty during simulation evaluation. The number of replicates is determined by raising the fidelity value to the power specified in the `fidelity_power` configuration parameter, rounded to the nearest integer above zero. Figure 21 provides an example in which a single design input value is provided while raising the fidelity input from one to ten across one-thousand simulation evaluations showing a convergence in fitness response. Figure 22 illustrates this effect while sweeping the design input from zero to one while raising the fidelity from one to ten across one-thousand simulation evaluations.

4.7 Semi-Cooperative Features

The simulation supports a variety of semi-cooperative features. These are features in which the simulation intentionally reports randomly incorrect results (Req. 65), to include feasibility or infeasibility (Req. 66), as well as fails to execute based on pre-determined fault injection thresholds (Req. 62). When the `enable_random_fitness` configuration option is enabled, the `random_fitness_rate` controls how often incorrect random fitness reports are returned from the simulation.

During scenario creation, some input dimensions are randomly selected to include fault injection with a corresponding failure threshold and magnitude. When a fault is present and the evaluated model response falls below the failure threshold the model response will be either degraded by scaling downward

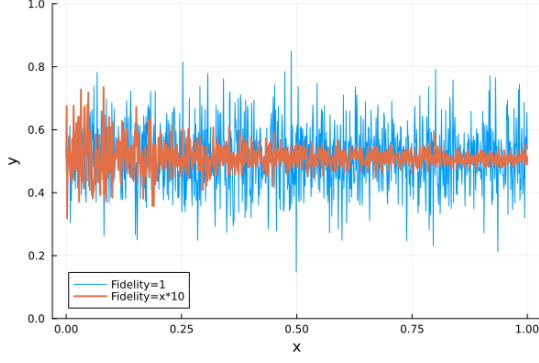


Figure 21: Demonstration of model fitness convergence by increasing model fidelity from one to ten.

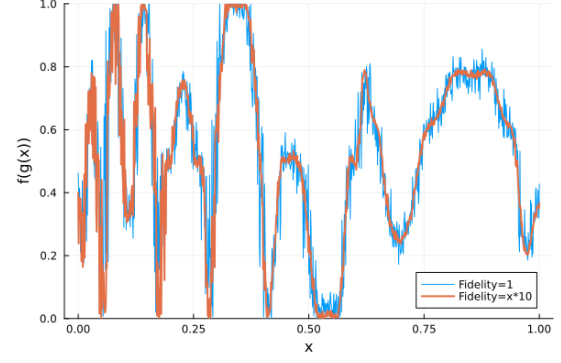


Figure 22: Example model response while increasing model fidelity from one to ten as design input variable x is increased from zero to one.

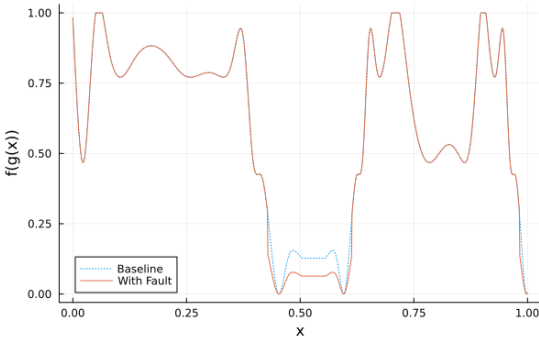


Figure 23: Example model fault resulting in degraded response.

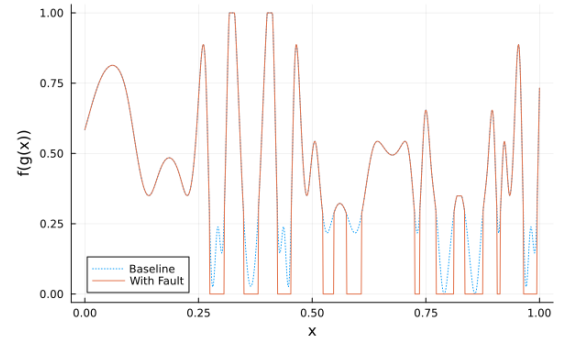


Figure 24: Example model executive failure triggered at locations where the model response falls below 0.29.

(Fig. 23) or will result in a simulation executive failure (Fig. 24) (Req. 61). In the event of an executive failure the simulation will abruptly stop and return 0.0 for the fitness score and will provide no indication that a failure was triggered. Fault via degradation and fault via executive failure result in separable and non-separable failure regions (Req. 63). Figure 25 provides an example composition of two dimensions, input noise, model noise, random failure and infeasibility, as well as large black executive failure regions.

4.8 Adversarial Features

The simulation creates an adversarial optimization environment as a function of the weighted history of fitness results, enabled using the `enable_adversarial` configuration option (Req. 20). The simulation object contains an `adversarial_fitness` state value that is continually updated each simulation evaluation by blending in the current iterations fitness value. The weighting of the blending is controlled via the `adversarial_feedback` parameter, resulting in a temporal memory of fitness with greater weight on the most recent solutions. The magnitude of the `adversarial_fitness` state value determines the scaling of adversarial features. This value is combined with a spline basic function to create non-linear artifacts in the scaling using the equation $x * (1 + (s(x) - 0.5) * 0.5)$ where x is the `adversarial_fitness` value in the range $[0,1]$ and $s(x)$ is the adversarial spline basic function evaluated at x (Figure 26).

The first method for adversarial feedback is by non-linear movement in the fitness landscape. if adversarial features are enabled in addition to the `enable_adversarial_offset` parameter, then each design input variables will be injected with a random bias scaled by the `adversarial_offset_mag` configuration parameter. This results in stronger movement in optimal basins if the recent fitness evaluations were near optimal. This effect is illustrated in Figure 27 and 28 by sweeping the design input from zero to one and then one to zero respectively. Different movements in the fitness landscapes are created based on recent fitness evaluations independent of what design inputs were used in the evaluations (Req. 59).

The second method for adversarial feedback is by scaling the frequency of random incorrect results, random failure and random infeasibility events, and input and model uncertainty (Req. 60). This is cre-

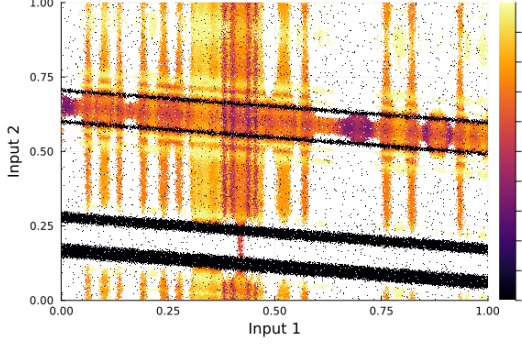


Figure 25: Example combining a two-dimensional composition function with input and model uncertainty, random failures and infeasibility reports, as well as executive failure regions.

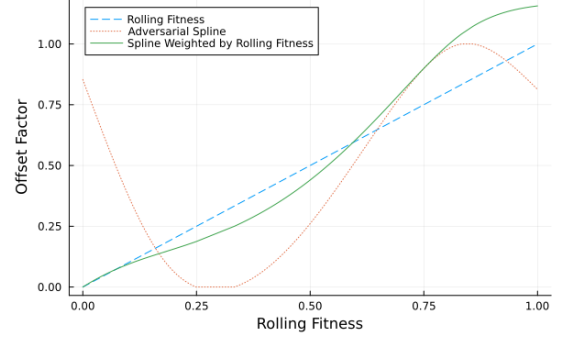


Figure 26: Construction of adversarial scaling factor by weighted sum of spline basic function and rolling fitness score.

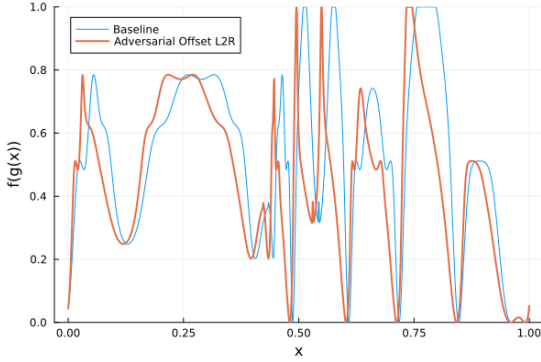


Figure 27: Application of adversarial scaling factor to model input bias while scanning input dimension x from zero to one (left to right) illustrating the temporal relationship in adversarial fitness memory.

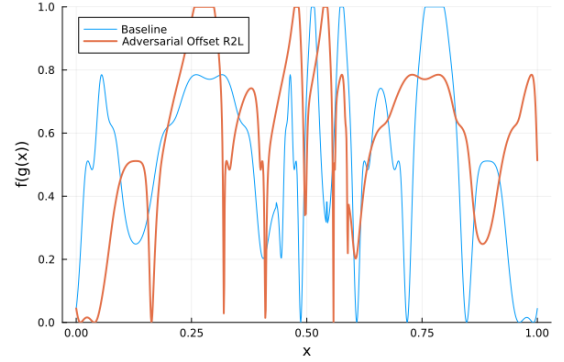


Figure 28: Application of adversarial scaling factor to model input bias while scanning input dimension x from one to zero (right to left) illustrating the temporal relationship in adversarial fitness memory.

ated by enabling the `enable_adversarial_incorrect`, `enable_adversarial_failure`, and `enable_adversarial_noise` configuration options respectively. The `adversarial_incorrect_factor`, `adversarial_failure_factor`, and `adversarial_noise_factor` configuration parameters increase the `random_fitness_rate`, `random_fault_rate`, and `random_infeasible_rate` frequencies by a multiple of $(1.0 + \text{factor} * \text{adversarial scaling})$ respectively. Figure 29 illustrates additional random failure events when the input value is near optimal around zero. Figure 30 illustrates the scaling in the uncertainty distribution by showing larger variance around the fitness solution as optimality is approached and equivalent uncertainty for fitness values near zero (Req. 53).

4.9 Network Pathfinding

The simulation capabilities discussed thus far apply to individual nodes. These nodes can be included in a scalable directed network where the number of nodes in the network is set in the simulation configuration object. Each node is visited in turn using an optimizer provided path array to the `sim_eval()` function (Req. 36). When the simulation is initialized, a nominal network path is created using a random permutation of the node numbers. Each node in the permutation has a valid exit linking it to the next node in the permutation creating a linear directed graph with one path through all of the nodes. Additional random directed links are created based on the `node_random_connection_freq` configuration parameter, as illustrated in Figure 31 (Req. 49).

The total fitness for the simulation evaluation is a weighted combination of the fitness function of each node that is visited (Req. 39). Node weighting is randomly set upon initialization and then additionally linearly scaled as a function of index in the nominal path such that successfully reaching later nodes

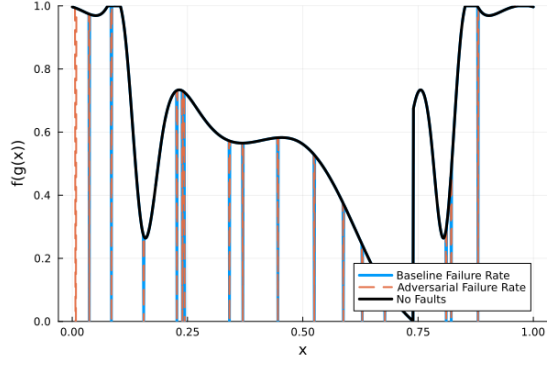


Figure 29: Example showing an elevation in random failure events due to adversarial response when the rolling fitness score is near optimal.

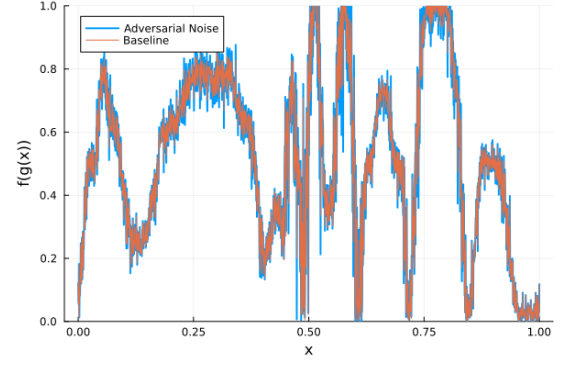


Figure 30: Example showing an elevation in input and model noise due to adversarial response when the rolling fitness score is near optimal.

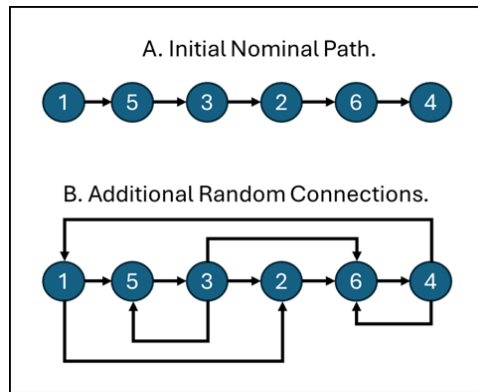


Figure 31: A: An example directed network single nominal path with six nodes. B: Random directed links added to the nominal path.

tends to have a higher impact on the overall fitness (Req. 41). The combination of continuous node design inputs with discrete node index pathfinding results in mixed-integer problems (Req. 11).

The simulation can enforce that certain nodes in the network are visited or else the run fails (Req. 42). This is enabled using the `enable_node_required_visits` option with number of required node visits determined using the `node_visit_required_rate` parameter. The rate parameter is multiplied by the number of nodes to set the first N nodes in the nominal path as required for visitation or else the simulation will return a failure fitness of 0.0. Lastly, if the optimizer provides an invalid path through the network, to include visiting a node multiple times, the simulation will stop crawling the network (Req. 40). At this point the simulation can return the current fitness or return a failure fitness of 0.0 if the option `enable_fail_on_bad_path` is set (Req. 64).

5 Configuration Parameter Tuning

Prior to running the performance comparison experiments, the simulation configuration parameters were tuned. As with the comparison experiments, ten replicates of thirty scenario seeds for ten-thousand iterations were used during each tuning cycle. Parameters were tuned using only the RND optimizer to allow features to have an observable impact without overly burdening or improving performance. The descriptions of parameters with setpoint justification is provided below in Table 3.

Before tuning, the following inputs were explicitly set as described in Table 3: model noise sigma, fidelity power, flattening rate, inversion rate, random fault rate, random infeasible rate, random fitness rate, constraint input margin, constraint optimal margin, min knots, max knots, min scale knots, max scale knots, min constraint knots, max constraint knots, max skew magnitude, knot feedback, adversarial noise factor, adversarial incorrect factor, adversarial feedback, min adversarial knots, max adversarial knots, dyn input map freq min, dyn input map freq max, dyn network map freq min, dyn network map freq max, dyn input shift freq min, dyn input shift freq max, node visit required freq, and node random connection freq.

Next, feature configuration parameters were tuned by running the three-million iteration experiment size using scenarios with five nodes of five dimensions. For each configuration parameter, a baseline F_{50} score was calculated with no effect, then bisection search was used to find a parameter setpoint that reached a specific F_{50} performance delta. This was chosen to balance the effect of different features on overall performance. After tuning, the parameters place the 90th percentile fitness (F_{90}) score just above 0.0 when all features are enabled in the four-dimension-four-node scenarios of Experiment 3 in the parent paper, thereby providing wide comparative illustration of the compounding effect of each feature on optimizer performance.

The first parameter tuned was the iso-optimal threshold. As the threshold is reduce from 1.0, the iso-optimal surface area grows making solutions easier to find. Bisection search was used to improve the F_{50} score to place it at 0.70. For the remaining parameters, the baseline score included the following enabled features: input mapping, input shifting, composition function scaling, node total fitness scaling, spline flattening, spline inversion, model noise, spline couple pair skewing, and node visitation requirements. The following parameters were tuned to create a $-0.02 \Delta F_{50}$: input noise sigma, dyn model inp vel mag, dyn model gov vel mag, dyn inp shift, adversarial offset mag, and adversarial failure factor. Adversarial features were tuned with only iso-optimal enabled to improve performance so that the adversarial effects are more visible. Lastly, due to the interaction between fault threshold and fault degradation rate, fault max thresh was first tuned to create a $-0.015 \Delta F_{50}$ with degradation rate set to 0.0. Then fault degradation rate was increased to create a $+0.005 \Delta F_{50}$, this way the combination of fault inputs resulted in an overall delta of only -0.01.

Table 3: Simulation Configuration Parameters.

Parameter	Default	Description	Justification
Scenario Options			
structure_seed	0	Sim structure RNG seed	Zero initialized
noise_seed	0	Run variation RNG seed	Zero initialized
num_dimensions	1	Number of dimensions at each node	Minimum valid value
num_nodes	1	Number of nodes in the network	Minimum valid value

Parameter	Default	Description	Justification
Feature Parameters			
input_noise_sigma	0.0016	Variation in decision variables	Achieve ΔF_{50} of -0.02 during tuning
model_noise_sigma	0.01	Variation in model outputs	Limit 3σ spread to 0.06
fidelity_power	1.5	Power of variable fidelity noise reduction	Desired convergence at fidelity level 10
flattening_rate	0.3	Probability of enabling flattening on splines	Support landscape variation
inversion_rate	0.3	Probability of enabling inversion on splines	Support landscape variation
random_fault_rate	0.02	Probability of returning a random fault result	To achieve 2% on average
random_infeasible_rate	0.02	Probability of returning a random infeasible result	To achieve 2% on average
random_fitness_rate	0.02	Probability of returning a random result	To achieve 2% on average
fault_degradation_rate	0.37	Probability of degradation instead of fault	Achieve ΔF_{50} of -0.010 during tuning
fault_max_thresh	0.77	Upper limit to fault threshold	Achieve ΔF_{50} of -0.015 during tuning
constraint_input_margin	0.05	Distance input must enter constraint to return infeasible	Large enough to prevent no feasible result
constraint_optimal_margin	0.05	Distance away from optimal before applying constraint	Large enough to prevent no optimal result
min_knots	10	Minimum knots for governing and model splines	Support landscape variation
max_knots	20	Maximum knots for governing and model splines	Support landscape variation
min_scale_knots	5	Minimum knots for scaling splines	Achieve non-linear scaling
max_scale_knots	10	Maximum knots for scaling splines	Achieve non-linear scaling
min_constraint_knots	10	Minimum knots for constraint splines	Support landscape variation
max_constraint_knots	20	Maximum knots for constraint splines	Support landscape variation
max_skew_magnitude	0.3	Maximum skew ratio for coupled model outputs	Meaningful reduction in axis alignment
knot_feedback	0.5	Temporal feedback in spline basic functions	Support landscape variation
adversarial_offset_mag	0.14	Magnitude of input shifting during adversarial feedback	Achieve ΔF_{50} of -0.02 during tuning
adversarial_noise_factor	3	Magnitude of noise amplification during adversarial feedback	Limit 3σ spread to 0.06
adversarial_failure_factor	0.33	Additional failure rate during adversarial feedback	Achieve ΔF_{50} of -0.02 during tuning
adversarial_incorrect_factor	3	Magnitude of random result amplification during adversarial feedback	Achieve 6% when near optimal
adversarial_feedback	0.8	Temporal feedback in fitness score for adversarial features	Demonstrated temporal memory

Parameter	Default	Description	Justification
min_adversarial_knots	5	Minimum knots for adversarial feature spline	Support non-linear spline landscape
max_adversarial_knots	10	Maximum knots for adversarial feature spline	Support non-linear spline landscape
dyn_model_inp_vel_mag	0.0077	Maximum input offset velocity magnitude per 1000 sim iterations	Achieve ΔF_{50} of -0.02 during tuning
dyn_model_gov_vel_mag	0.0077	Maximum governing offset velocity magnitude per 1000 sim iterations	Achieve ΔF_{50} of -0.02 during tuning
dyn_input_map_freq_min	1050	Minimum iterations before input re-mapping event	Achieve 6 to 9 events per 10k iterations
dyn_input_map_freq_max	1600	Maximum iterations before input re-mapping event	Achieve 6 to 9 events per 10k iterations
dyn_network_map_freq_min	1050	Minimum iterations before network re-mapping event	Achieve 6 to 9 events per 10k iterations
dyn_network_map_freq_max	1600	Maximum iterations before network re-mapping event	Achieve 6 to 9 events per 10k iterations
dyn_input_shift_freq_min	1450	Minimum iterations before input offset shift event	Achieve 3 to 5 events per 10k iterations
dyn_input_shift_freq_max	1700	Maximum iterations before input offset shift event	Achieve 3 to 5 events per 10k iterations
dyn_input_shift_mag	0.0071	Maximum offset during input shift event	Achieve ΔF_{50} of -0.02 during tuning
node_visit_required_freq	0.1	Frequency of selecting a node for required visitation	Minimally demonstrated effect
node_random_connection_freq	0.2	Frequency of adding extra random node connections	Support non-linear node networks
Feature Enablement			
enable_input_map	TRUE	Enable the input mapping feature	Max difficulty by default
enable_input_shift	TRUE	Enable the input shifting feature	Max difficulty by default
enable_input_noise	TRUE	Enable the input noise feature	Max difficulty by default
enable_model_noise	TRUE	Enable the model noise feature	Max difficulty by default
enable_adversarial	TRUE	Enable adversarial features	Max difficulty by default
enable_adversarial_noise	TRUE	Enable adversarial noise feature	Max difficulty by default
enable_adversarial_offset	TRUE	Enable adversarial offset feature	Max difficulty by default
enable_adversarial_failure	TRUE	Enable adversarial failure feature	Max difficulty by default
enable_adversarial_incorrect	TRUE	Enable adversarial incorrect feature	Max difficulty by default
enable_dynamic	TRUE	Enable dynamic feature	Max difficulty by default
enable_dynamic_spline	TRUE	Enable dynamic spline feature	Max difficulty by default
enable_dynamic_input_map	TRUE	Enable dynamic input map feature	Max difficulty by default
enable_dynamic_input_shift	TRUE	Enable dynamic input shift feature	Max difficulty by default
enable_dynamic_network	TRUE	Enable dynamic network feature	Max difficulty by default

Parameter	Default	Description	Justification
enable_constraints	TRUE	Enable feasibility constraints feature	Max difficulty by default
enable_flattening	TRUE	Enable spline flattening feature	Max difficulty by default
enable_faults	TRUE	Enable fault features	Max difficulty by default
enable_inversion	TRUE	Enable spline inversion feature	Max difficulty by default
enable_iso_optimal	TRUE	Enable iso optimal surface feature	Max difficulty by default
enable_skewing	TRUE	Enable coupled response skewing feature	Max difficulty by default
enable_comp_scale_inputs	TRUE	Enable composition function scaling inputs	Max difficulty by default
enable_total_scale_input	TRUE	Enable total node fitness scaling inputs	Max difficulty by default
enable_random_fitness	TRUE	Enable random fitness result feature	Max difficulty by default
enable_node_required_visits	TRUE	Enable node visitation required feature	Max difficulty by default
enable_fail_on_bad_path	TRUE	Enable executive failure on invalid network path	Max difficulty by default
quiet_init	FALSE	Print simulation construction log to console	Reduce console output by default
save_debug_figs	FALSE	Save simulation construction debug figures	Not needed for normal runs

References

- [1] “The Crucible Simulation GitHub Repository.” Website: https://github.com/kennedydm/crucible_sim/.
- [2] J. Liang and K. Deb, “Problem Definitions and Evaluation Criteria for the CEC 2005 Special Session on Real-Parameter Optimization,” 2005.
- [3] B. Addis and M. Locatelli, “A new class of test functions for global optimization,” *J Glob Optim*, vol. 38, pp. 479–501, July 2007.
- [4] J. J. Liang, B. Y. Qu, P. N. Suganthan, and A. G. Hernández-Díaz, “Problem Definitions and Evaluation Criteria for the CEC 2013 Special Session on Real-Parameter Optimization,” 2013.
- [5] A. Ahrari, S. Elsayed, R. Sarker, D. Essam, and C. Coello, *Problem Definition and Evaluation Criteria for the CEC’2022 Competition on Dynamic Multimodal Optimization*. Jan. 2022.
- [6] A. Kumar, K. V. Price, A. W. Mohamed, and A. A. Hadi, “Problem Definitions and Evaluation Criteria for the CEC 2022 Special Session and Competition on Single Objective Bound Constrained Numerical Optimization,” 2022.
- [7] J. Liang, C. Yue, G. Li, B. Qu, P. Suganthan, and K. Yu, *Problem Definitions and Evaluation Criteria for the CEC 2021 on Multimodal Multiobjective Path Planning Optimization*. Dec. 2020.
- [8] J. R. R. A. Martins and A. Ning, *Engineering Design Optimization*. Cambridge University Press, 1 ed., Nov. 2021.
- [9] K. Tang, X. Yao, P. N. Suganthan, C. MacNish, Y. P. Chen, C. M. Chen, and Z. Yang, “Benchmark Functions for the CEC’2008 Special Session and Competition on Large Scale Global Optimization,” 2008.
- [10] Q. Zhang, A. Zhou, S. Zhao, P. Suganthan, W. Liu, and S. Tiwari, “Multiobjective optimization Test Instances for the CEC 2009 Special Session and Competition,” *Mechanical Engineering*, 2009.

- [11] X. Li, A. Engelbrecht, and M. G. Epitropakis, “Benchmark Functions for CEC’2013 Special Session and Competition on Niching Methods for Multimodal Function Optimization,” 2013.
- [12] G. Wu, R. Mallipeddi, and P. Suganthan, *Problem Definitions and Evaluation Criteria for the CEC 2017 Competition and Special Session on Constrained Single Objective Real-Parameter Optimization*. Oct. 2016.
- [13] K. V. Price, N. H. Awad, M. Z. Ali, and P. N. Suganthan, “Problem Definitions and Evaluation Criteria for the 100-Digit Challenge Special Session and Competition on Single Objective Numerical Optimization,” 2018.
- [14] C. T. Yue, K. V. Price, P. N. Suganthan, J. J. Liang, M. Z. Ali, B. Y. Qu, N. H. Awad, and P. P. Biswas, “Problem Definitions and Evaluation Criteria for the CEC 2020 Special Session and Competition on Single Objective Bound Constrained Numerical Optimization,” 2020.
- [15] V. Beiranvand, W. Hare, and Y. Lucet, “Best practices for comparing optimization algorithms,” *Optim Eng*, vol. 18, pp. 815–848, Dec. 2017.
- [16] F. Di Fiore, M. Nardelli, and L. Mainini, “Active Learning and Bayesian Optimization: A Unified Perspective to Learn with a Goal,” *Arch Computat Methods Eng*, vol. 31, pp. 2985–3013, July 2024.
- [17] W. Luo, X. Lin, C. Li, and Y. Shi, *Benchmark Functions for CEC 2022 Competition on Seeking Multiple Optima in Dynamic Environments*. Jan. 2022.
- [18] V. A. W. J. Marchau, W. E. Walker, P. J. T. M. Bloemen, and S. W. Popper, *Decision Making under Deep Uncertainty: From Theory to Practice*. Cham: Springer International Publishing, 2019.
- [19] T. Ören, B. P. Zeigler, and A. Tolk, *Body of Knowledge for Modeling and Simulation: A Handbook by the Society for Modeling and Simulation International*. Simulation Foundations, Methods and Applications, Cham: Springer International Publishing, 2023.
- [20] M. C. Fu, *Handbook of Simulation Optimization*, vol. 216 of *International Series in Operations Research & Management Science*. New York, NY: Springer New York, 2015.
- [21] R. Mallipeddi and P. Suganthan, *Problem definitions and evaluation criteria for the CEC 2010 Competition on Constrained Real-Parameter Optimization*. May 2010.
- [22] J. Liang, T. Runarsson, E. Mezura-Montes, M. Clerc, P. Suganthan, C. Coello, and K. Deb, “Problem definitions and evaluation criteria for the CEC 2006 special session on constrained real-parameter optimization,” *Nanyang Technological University, Singapore, Tech. Rep*, vol. 41, Jan. 2006.
- [23] C. Li, S. Yang, T. T. Nguyen, E. L. Yu, X. Yao, and Y. Jin, “Benchmark Generator for CEC’2009 Competition on Dynamic Optimization,” 2008.
- [24] S. Huband, L. Barone, L. While, and P. Hingston, “A Scalable Multi-objective Test Problem Toolkit,” in *Evolutionary Multi-Criterion Optimization*, (Berlin, Heidelberg), pp. 280–295, Springer, 2005.
- [25] J. Liang, P. Suganthan, B. Qu, D. Gong, and C. Yue, *Problem Definitions and Evaluation Criteria for the CEC 2020 Special Session on Multimodal Multiobjective Optimization*. Dec. 2019.
- [26] C. A. Floudas and P. M. Pardalos, *Encyclopedia of optimization*. Springer reference, New York: Springer, 2nd ed ed., 2009.
- [27] N. Hansen, A. , Anne, R. , Raymond, M. , Olaf, T. , Tea, , and D. Brockhoff, “COCO: a platform for comparing continuous optimizers in a black-box setting,” *Optimization Methods and Software*, vol. 36, pp. 114–144, Jan. 2021.
- [28] T. Dokeroglu, T. Kucukyilmaz, and E.-G. Talbi, “Hyper-heuristics: A survey and taxonomy,” *Computers & Industrial Engineering*, vol. 187, p. 109815, Jan. 2024.
- [29] “The Julia Programming Language.” Website: <https://julialang.org/>.